

UNIVERSITÉ LAVAL

Faculté de sciences et génies

Planification de Maintenance Ferroviaire : Comparaison de Deux Modèles MiniZinc sur Chuffed et Gecode

IFT-4001 IFT-7020

Présenté à
Claude-Guy Quimper

par

Ethan Facca, Léa Potin

Matricule	**Nom**
537 403 134	Ethan Facca
537 396 798	Léa Potin

Table 1 – Matricules et noms des membres de l'équipe

14 décembre 2025

1 Introduction

La planification de maintenance ferroviaire représente un défi opérationnel majeur pour les entreprises de transport, notamment la SNCF. Avec des centaines de trains nécessitant des maintenances régulières, des fenêtres temporelles strictes, et des ressources limitées (voies, techniciens, équipements), ce problème présente toutes les caractéristiques d'un problème d'optimisation combinatoire complexe.

Ce projet s'intéresse à la modélisation de ce problème comme un **Resource-Constrained Project Scheduling Problem (RCPSP)** avec des contraintes spécifiques au domaine ferroviaire : fenêtres temporelles strictes, temps de setup variables selon le contexte, et ressources multiples avec capacités simultanées.

1.1 Problématique

Le problème consiste à planifier les opérations de maintenance sur un horizon de 24 heures en respectant :

— Les fenêtres temporelles d'arrivée et de départ des trains
— Les contraintes de précédence entre opérations
— Les capacités limitées des ressources (voies, techniciens, équipements)
— Les temps de setup entre opérations et entre trains différents
— Les compatibilités entre voies et équipements

L'objectif est de minimiser le retard pondéré par priorité des maintenances, où les maintenances urgentes ont un poids plus élevé que les préventives.

1.2 Objectifs

Les objectifs de ce projet sont :

1. Modéliser le problème de planification de maintenance ferroviaire comme un RCPSP avec contraintes temporelles
2. Développer deux modèles MiniZinc distincts avec des approches de modélisation différentes
3. Comparer les performances des deux modèles sur deux solveurs (Chuffed et Gecode) sur différentes instances
4. Identifier les limites de résolubilité selon la complexité des instances
5. **Découvrir** l'impact de différentes modélisations sur la performance des solveurs et comparer le comportement de Chuffed et Gecode

1.3 Structure du rapport

Ce rapport est organisé comme suit : la Section 2 présente la description formelle du problème, la Section 3 décrit les deux modèles développés, la Section 4 présente le protocole expérimental, la Section 5 expose les résultats expérimentaux, la Section 6 en discute les implications, et la Section 7 conclut le travail.

2 Description du problème

Cette section présente le problème de planification de maintenance ferroviaire. Le problème consiste à planifier les opérations de maintenance sur un horizon de 24 heures en respectant les contraintes de ressources et les fenêtres temporelles des trains.

2.1 Paramètres d'entrée

Une instance du problème contient les éléments suivants :

Trains : Chaque train t a un temps d'arrivée a_t et un temps de départ d_t (en minutes). Les maintenances doivent être effectuées pendant que le train est présent à la gare.

Maintenances : Chaque maintenance m est associée à un train, a une priorité (1 = urgente, 2 = préventive) et une date butoir idéale δ_m. L'objectif est de minimiser les retards par rapport à cette date butoir, en donnant plus d'importance aux maintenances urgentes.

Opérations : Chaque maintenance est composée de plusieurs opérations. Chaque opération o a une durée dur_o, peut avoir un prédécesseur (contrainte de précédence), et nécessite une voie spécifique et des ressources (techniciens et équipements).

Ressources :

— **Techniciens :** Il y a plusieurs types de techniciens. Chaque type k a un nombre limité N_k de techniciens disponibles, et chaque technicien peut travailler au maximum $T_{max,k}$ minutes.

— **Équipements :** Il y a plusieurs types d'équipements. Chaque type e a une capacité simultanée maximale C_e (nombre d'opérations qui peuvent utiliser cet équipement en même temps).

— **Voies :** Les opérations doivent être assignées à des voies. Certains équipements ne sont disponibles que sur certaines voies.

Setup : Il y a des temps de setup entre opérations (σ_{op} minutes) et entre trains différents (σ_{train} minutes, plus long car il faut changer de train).

2.2 Variables de décision

Pour résoudre le problème, nous devons décider :

— Le temps de début s_o et de fin f_o de chaque opération o

— Le temps de début s_m et de fin f_m de chaque maintenance m

— La voie x_o assignée à chaque opération o

2.3 Contraintes

Une solution est valide si elle respecte toutes les contraintes suivantes :

Cohérence temporelle : La fin d'une opération est égale à son début plus sa durée : $f_o = s_o + dur_o$. Le début d'une maintenance est le minimum des débuts de ses opérations, et sa fin est le maximum des fins de ses opérations.

Fenêtres temporelles : Toutes les opérations d'une maintenance doivent être effectuées pendant que le train est présent : $a_t \leq s_m \leq f_m \leq d_t$ où t est le train associé à la maintenance m.

Précédence : Si une opération a un prédécesseur, elle doit commencer après la fin du prédécesseur plus le temps de setup : $s_o \geq f_{pred(o)} + \sigma_{op}$.

Non-chevauchement sur les voies : Deux opérations ne peuvent pas utiliser la même voie en même temps. Si elles sont du même train, il faut un setup de σ_{op} minutes entre elles. Si elles sont de trains différents, il faut un setup de σ_{train} minutes.

Capacité des ressources : À tout moment, le nombre de techniciens de chaque type utilisés ne doit pas dépasser le nombre disponible. De même, le nombre d'équipements de chaque type utilisés simultanément ne doit pas dépasser la capacité maximale.

Compatibilité voie-équipement : Si une opération nécessite un équipement, elle doit être assignée à une voie où cet équipement est disponible.

Limite de charge de travail : La charge totale de travail (somme des durées × nombre de techniciens) pour chaque type de technicien ne doit pas dépasser la capacité totale disponible.

2.4 Fonction objectif

L'objectif est de minimiser le retard pondéré total : $\min \sum_m p_m \cdot \max(0, f_m - \delta_m)$ où p_m est la priorité de la maintenance m et δ_m est sa date butoir. Les maintenances urgentes (priorité 1) ont un poids plus élevé que les préventives (priorité 2).

3 Approche proposée

Cette section présente la modélisation du problème en programmation par contraintes avec MiniZinc. Deux modèles distincts ont été développés, chacun avec une approche différente pour modéliser les contraintes de ressources et les non-chevauchements.

3.1 Variables de décision

Les deux modèles utilisent les mêmes variables de décision :
— `start_op[o]` : Temps de début de l'opération o (domaine : $[0, H]$ où $H = 1440$ minutes)
— `end_op[o]` : Temps de fin de l'opération o (domaine : $[0, H]$)
— `start_maint[m]` : Temps de début de la maintenance m (domaine : $[0, H]$)
— `end_maint[m]` : Temps de fin de la maintenance m (domaine : $[0, H]$)
— `assigned_track[o]` : Voie assignée à l'opération o (domaine : ensemble des voies disponibles)

3.2 Modèle 1 :

[À compléter avec la description du Modèle 1]

3.3 Modèle 2 : Approche alternative

3.4 Contraintes principales

Les deux modèles implémentent toutes les contraintes décrites dans la Section 2. La différence principale réside dans la modélisation des contraintes de ressources et des non-chevauchements.

Le Modèle 2 utilise une approche différente pour modéliser les contraintes de ressources. Les contraintes les plus importantes sont :

Capacité des ressources : Pour vérifier que les ressources ne sont jamais dépassées, le modèle vérifie la capacité à chaque instant où une opération commence. Pour chaque opération o et chaque type de technicien k, on somme le nombre de techniciens utilisés par toutes les opérations qui sont en cours au moment où o commence. Cette somme ne doit pas dépasser le nombre de techniciens disponibles N_k. La même logique s'applique aux équipements avec leur capacité simultanée C_e.

Non-chevauchement : Pour deux opérations qui utilisent la même voie, le modèle impose qu'elles ne se chevauchent pas. Si elles sont du même train, il faut au moins σ_{op} minutes entre la fin de l'une et le début de l'autre. Si elles sont de trains différents, il faut au moins σ_{train} minutes.

Heuristiques : Le modèle utilise les heuristiques par défaut des solveurs MiniZinc. Chuffed utilise des techniques de propagation de contraintes et de recherche locale, tandis que Gecode utilise des stratégies de branchement (first-fail, domain splitting). Aucune heuristique personnalisée n'a été implémentée.

3.5 Fonction objectif

L'objectif est de minimiser le retard pondéré total : $\min \sum_m p_m \cdot \max(0, f_m - \delta_m)$ où p_m est la priorité et δ_m la date butoir de la maintenance m.

4 Protocole d'expérimentation

Cette section décrit le protocole expérimental pour comparer les deux modèles développés sur deux solveurs MiniZinc (Chuffed et Gecode).

4.1 Instances du banc d'essai

Huit instances de test ont été générées automatiquement avec un script Python, couvrant différentes tailles :

— **Small :** 2 instances (small_01 : 2 trains, 8 opérations ; small_02 : 3 trains, 10 opérations)
— **Medium :** 3 instances (30 à 45 opérations, 5 à 7 trains)
— **Large :** 3 instances (60 à 90 opérations, 10 à 15 trains)

Toutes les instances ont un horizon de 24 heures (1440 minutes), des temps de setup de 10 minutes entre opérations et 30 minutes entre trains différents, et un mélange de maintenances urgentes et préventives.

4.2 Métriques

Pour chaque instance et chaque solveur, nous mesurons :

— **Temps de résolution :** Temps total pour trouver une solution ou prouver l'insatisfiabilité (en secondes)
— **Statut :** Résolu, insatisfiable (UNSAT), ou timeout (> 300 secondes)
— **Qualité de la solution :** Valeur de l'objectif (retard pondéré) pour les instances résolubles

4.3 Environnement

Les tests sont effectués sur une machine Windows 11 avec un processeur Intel i7-11800H et 16 Go de RAM. MiniZinc version 2.9.4 est utilisé avec Chuffed 0.13.2 et Gecode 6.3.0. Le temps limite est fixé à 300 secondes par instance. Pour chaque combinaison instance-modèle-solveur (8 instances $\times$ 2 modèles $\times$ 2 solveurs = 32 combinaisons), les métriques sont collectées et enregistrées.

5 Résultats

Cette section présente les résultats expérimentaux de la comparaison des deux modèles sur Chuffed et Gecode.

5.1 Performance globale

Le Tableau 2 présente les temps de résolution pour chaque combinaison instance-modèle-solveur. Les timeouts (dépassement du temps limite de 300 secondes) sont indiqués par **TO**.

Table 2 – Temps de résolution (secondes) pour chaque combinaison instance-modèle-solveur

Instance	Modèle 1 (Chuffed)	Modèle 1 (Gecode)	Modèle 2 (Chuffed)	Modèle 2 (Gecode)
small_01	[À compléter]	[À compléter]	0.6	0.3
small_02	[À compléter]	[À compléter]	1.9	4.8
medium_01	[À compléter]	[À compléter]	**UNSAT** (1.5s)	**UNSAT** (1.4s)
medium_02	[À compléter]	[À compléter]	**UNSAT** (2.6s)	**TO**
medium_03	[À compléter]	[À compléter]	**UNSAT** (4.0s)	**TO**
large_01	[À compléter]	[À compléter]	**UNSAT** (5.9s)	**UNSAT** (26.5s)
large_02	[À compléter]	[À compléter]	**UNSAT** (14.1s)	**UNSAT** (12.8s)
large_03	[À compléter]	[À compléter]	**UNSAT** (13.5s)	**UNSAT** (12.6s)

Note : TO = Timeout ($>$ 300s), UNSAT = Instance insatisfiable (aucune solution n'existe). Les valeurs entre parenthèses indiquent le temps nécessaire pour prouver l'insatisfiabilité.

5.2 Comparaison des modèles

5.3 Comparaison des solveurs

5.4 Observations principales

6 Discussion

Cette section discute les résultats expérimentaux et présente les découvertes concernant le comportement des solveurs Chuffed et Gecode.

6.1 Performance des solveurs

Les résultats montrent des différences importantes entre Chuffed et Gecode. Sur les instances medium_02 et medium_03, Chuffed prouve rapidement (en 2.6s et 4.0s) que ces instances sont

insatisfiables (aucune solution n'existe), tandis que Gecode timeout après 300 secondes sans pouvoir conclure. Cela suggère que Chuffed est plus efficace pour détecter rapidement les conflits de contraintes.

Sur les instances résolubles (small_01 et small_02), Gecode trouve des solutions de meilleure qualité : objectif de 0 vs 514 pour small_01, et 3336 vs 4564 pour small_02. Cela suggère que Gecode explore l'espace de recherche de manière plus systématique pour trouver des solutions optimales, tandis que Chuffed se contente de solutions réalisables plus rapidement.

6.2 Facteurs de difficulté

Les résultats montrent un pattern clair : les instances small (8-12 opérations) sont résolubles, tandis que les instances medium et large sont principalement insatisfiables. Plusieurs facteurs expliquent cette observation :

— Les fenêtres temporelles peuvent être trop serrées pour permettre une planification satisfaisante avec beaucoup d'opérations
— Le ratio ressources/demande peut être insuffisant pour les instances plus grandes
— Les contraintes de précédence et de compatibilité voie-équipement peuvent créer des conflits impossibles à résoudre

6.3 Limites et perspectives

Les limites de cette étude incluent le nombre limité d'instances (8) et seulement deux solveurs testés. Les perspectives futures incluent tester d'autres solveurs (OR-Tools, CP-SAT, etc.) et générer plus d'instances avec différentes caractéristiques pour mieux comprendre les facteurs de difficulté.

7 Conclusion

Ce projet a permis de développer et comparer deux modèles distincts pour le problème de planification de maintenance ferroviaire, évalués sur deux solveurs MiniZinc (Chuffed et Gecode).

Les contributions principales sont : (1) le développement de deux modèles MiniZinc avec des approches différentes, (2) l'évaluation systématique sur 8 instances, (3) la comparaison des performances des deux modèles et des deux solveurs, et (4) l'identification des facteurs de difficulté.

La découverte principale est que Chuffed et Gecode ont des forces complémentaires : Chuffed excelle pour détecter l'insatisfiabilité rapidement (2.6s vs timeout), tandis que Gecode trouve des solutions de meilleure qualité sur les instances résolubles. Les résultats montrent aussi que la majorité des instances medium et large sont insatisfiables, suggérant que les contraintes sont très strictes pour ces configurations.

Les perspectives futures incluent tester d'autres solveurs et générer plus d'instances avec différentes caractéristiques pour mieux comprendre l'impact des choix de modélisation.

A Annexe : Code

A.1 Modèle MiniZinc

Le code complet du modèle MiniZinc est disponible dans le fichier `src/model.mzn`.

A.2 Exemple de données

Un exemple de fichier de données est fourni dans `src/data_example.dzn`.